

FATOORA · SAUDI E-INVOICING

ZATCA Phase 2 Integration Guide

A complete, end-to-end reference for integrating ZATCA (Fatoora) Phase 2 e-invoicing into a POS/retail system — onboarding, invoice signing, QR codes, checkout, and how a single VAT-registered legal entity with multiple branches should be modeled.

Prepared 2026-07-22 · Reference implementation: Mimony Mart / baqala-bright-flow (.NET API)

Contents

1. What ZATCA Phase 2 actually requires
2. Prerequisites & environments (sandbox / simulation / production)
3. Onboarding flow: CSR → OTP → Compliance CSID → Compliance Tests → Production CSID
4. Multi-branch architecture: one VAT/CR number, many branches
5. Data model recommendation
6. Invoice generation & signing pipeline
7. QR code generation (TLV tags)
8. Checkout integration & the reporting/clearance split
9. API endpoint reference
10. Gotchas & lessons learned (the expensive ones)
11. Frontend integration notes
12. Build checklist for the new project

1 What ZATCA Phase 2 actually requires

ZATCA ("Zakat, Tax and Customs Authority") Phase 2 — also called "Integration Phase" — requires every VAT-registered business in Saudi Arabia to generate invoices as structured XML (UBL 2.1), cryptographically sign them, embed a QR code, and transmit them to ZATCA's government gateway in near-real-time. This replaces Phase 1, which only required a QR code with no signing or transmission.

CONCEPT	MEANING
EGS	Electronic Generation Solution — the software unit ZATCA registers. Each EGS gets its own cryptographic identity (CSR/certificate/CSID).
CSID	Cryptographic Stamp Identifier — the X.509 certificate + secret ZATCA issues after validating a CSR. There are two: Compliance CSID (sandbox testing only) and Production CSID (real invoices).
Simplified invoice (B2C)	No buyer VAT number captured (retail/POS sale to a consumer). Must be signed, QR'd, and reported to ZATCA within 24 hours — reporting happens after the invoice is delivered to the customer.
Standard invoice (B2B)	Buyer VAT number known. Must be cleared by ZATCA <i>before</i> it can be delivered to the buyer — a real-time, blocking round trip.
ICV / PIH	Invoice Counter Value (a strictly incrementing integer) and Previous Invoice Hash — every invoice under one CSID chains to the previous one's hash, forming a tamper-evident sequence. Break the chain and every subsequent invoice fails validation.

Reference implementation used in this guide

Everything below is grounded in a real, working native .NET port (no PHP/Node sidecar) that has been verified against ZATCA's live sandbox gateway with genuine 200 ISSUED responses. File paths referenced throughout (e.g. `api/Services/ZatcaService.cs`) point at that implementation so you can read the actual code, not just this description of it.

2 Prerequisites & environments

- A valid **15-digit VAT registration number** and **Commercial Registration (CR) number** for the legal entity.
- Access to the business's **Fatoora portal** account (ZATCA's onboarding UI) — this is where a human operator pastes the CSR to receive an OTP. This step cannot be automated; ZATCA requires a human in the loop for OTP issuance.
- A decision on **industry category** and **invoice type support** (standard only, simplified only, or both — encoded as a 4-digit bitmask, e.g. 1100 = supports both).

ENVIRONMENT	GATEWAY PATH SEGMENT	PURPOSE
Sandbox (NonProduction)	developer-portal	Free-form testing; certificate template name TSTZATCA-Code-Signing .
Simulation	simulation	Pre-production dress rehearsal with realistic data; template PREZATCA-Code-Signing .
Production	production	Live invoices, real legal effect; template ZATCA-Code-Signing .

All three share the same host: `https://gw-fatoora.zatca.gov.sa/e-invoicing`. Every request needs headers `Accept-Version: V2`, `Accept-Language: en`, and a bare `Content-Type: application/json` (ZATCA's gateway rejects a charset parameter on this header — a small but real footgun).

3 Onboarding flow

Onboarding is a strict five-step sequence. Steps 1 and 5 are pure local cryptography or storage; steps 2–4 are round trips to ZATCA's gateway. Nothing in step 3 onward can succeed if step 1's CSR fields are wrong — and ZATCA's rejections at that stage are frustratingly generic (see §10).

1 Generate a keypair & CSR (local, no network call)

Generate an EC keypair (ZATCA requires `secp256k1`). Build a CSR whose Subject and SAN carry ZATCA-specific fields — this is the part every implementation gets subtly wrong on the first try. See the exact field table below.

CSR FIELD	VALUE	NOTES
Subject DN order	<code>C, OU, O, CN</code>	Must match this order — ZATCA's own sample <code>openssl.cnf</code> is order-sensitive.
<code>C</code> (Country)	<code>SA</code>	Hardcoded.
<code>OU</code> (Organizational Unit)	Branch / site name	This is where the branch identity belongs — see §4.
<code>O</code> (Organization)	Legal company name	Same across all branches.
<code>CN</code> (Common Name)	EGS serial number	Format <code>1-{' '}SolutionName{' '} 2-{' '}Model{' '} 3-{' '}UUID{' '}</code> , e.g. <code>1-MySolution 2-EGS 3-<uuid></code> .
SAN <code>SN</code>	Same EGS serial as CN	Must use OID 2.5.4.4 (the OpenSSL short-name "SN" actually means <i>surname</i>). Using OID 2.5.4.5 (the real <code>serialNumber</code> OID) produces a byte-length-identical CSR that ZATCA silently rejects. See §10.
SAN <code>UID</code>	VAT registration number	OID <code>0.9.2342.19200300.100.1.1</code> — not <code>...100.1.25</code> (that OID is <code>domainComponent</code> , a different field entirely).
SAN <code>T</code> (title)	Invoice-type bitmask	e.g. <code>1100</code> .
SAN <code>registeredAddress</code>	Branch city/address	OID <code>2.5.4.26</code> .
SAN <code>businessCategory</code>	Industry, e.g. "Retail Trade"	Free text, must be non-empty.
Private extension	<code>certificateTemplateName</code>	OID <code>1.3.6.1.4.1.311.20.2</code> ; value depends on environment (sandbox/simulation/production template names above).
Signature algorithm	<code>SHA256withECDSA</code>	

Why this table is this detailed

Every field above is a place a "structurally valid" CSR can still be rejected. .NET's built-in `CertificateRequest` cannot express the required `directoryName` SAN at all — you need a library that lets you hand-construct ASN.1 (BouncyCastle in the .NET reference; equivalent low-level ASN.1 control in whatever stack the new project uses).

2 Human operator submits the CSR to the Fatoora portal, receives an OTP

This step is manual and out-of-band — there is no API to request the OTP. The CSR (base64) is copied into ZATCA's Fatoora onboarding UI by whoever administers the account; ZATCA sends back a one-time password.

3 Redeem the OTP for a Compliance CSID

POST `/[env]/compliance` · header `OTP: <otp>` · body `{'{' "csr": "<base64 CSR>" {'}'}`

Response contains `requestID`, `binarySecurityToken` (the compliance certificate) and `secret`. Store all three — they're the Basic Auth credentials for every subsequent sandbox call.

4 Run the six mandatory compliance test documents

POST `/[env]/compliance/invoices` · Basic Auth = `binarySecurityToken:secret`

ZATCA requires exactly six signed sample documents to be submitted and accepted before it will issue a Production CSID:

#	TYPE CODE	SUBTYPE	MEANING
1	388	0100000	Standard Invoice
2	381	0100000	Standard Credit Note
3	383	0100000	Standard Debit Note
4	388	0200000	Simplified Invoice
5	381	0200000	Simplified Credit Note
6	383	0200000	Simplified Debit Note

Each is signed and chained via ICV (1–6, seeded from the well-known empty-invoice hash `NWZLY2ViNjZmZmM4NmYzOGQ5NTI3ODZjNmQ2OTZjNzljMmRiYzIzOWRkNGU5MWI0NjcyOWQ3M2EyN2ZiNTdlOQ==`). Pass/fail is read from `reportingStatus/clearanceStatus` in the response (values `REPORTED` / `CLEARED`), or a "compliance check already completed" message treated as an already-passed shortcut on retry. All six must pass before proceeding.

5 Request the Production CSID

POST `/[env]/production/csids` · body `{'{' "compliance_request_id": <step 3's requestID> {'}'}` · Basic Auth = `compliance CSID/secret`

Store the returned `requestID`/`binarySecurityToken`/`secret` as the **production** credentials. Because this is a new certificate/device, **reset the ICV to 0 and the invoice hash back to the seed value** — the production CSID starts its own fresh hash chain, independent of the six compliance-test documents.

Track onboarding status explicitly

Persist a simple state field — `not_started` → `csr_generated` → `compliance_csid_obtained` → `production_ready` — and gate every downstream feature (real invoice submission) on `production_ready`. This makes retries idempotent: if step 4 partially fails, you can re-run from wherever it stopped instead of starting over.

4 Multi-branch architecture: one VAT/CR number, many branches

This is the question that most needs a deliberate decision, because ZATCA's own model and the "simplest to build" model pull in different directions. A single legal entity (one CR/VAT number, one Fatoora account, one owner/admin) commonly operates several physical branches. ZATCA's intended model treats **each branch as its own EGS "device"** with its own CSR/CSID and its own invoice hash chain — while all of them share the same VAT number and legal identity in the CSR's UID/0 fields.

Model A — Shared Singleton Identity (what the reference implementation actually does)

- One CSR/CSID/keypair/hash-chain for the *entire* company, stored in a single singleton row.
- Every branch's checkout submits invoices under the same certificate, incrementing the same ICV/PIH chain.
- Concurrent submissions from different branches must be serialized (row-lock the singleton, wrap the ZATCA call in a DB transaction) to avoid forking the hash chain.
- **Simple to build**, but not what ZATCA's Developer Portal Manual actually models — and a single point of failure: one certificate revocation or hash-chain corruption takes down invoicing for every branch at once.
- Branch-level onboarding UI (a wizard per branch) is **misleading** under this model — the second branch's "CSR generation" silently overwrites the first branch's CSR/keypair on the same shared row.

Model B — Per-Branch CSID (recommended for a new build)

- Each branch generates its **own** CSR (its own EC keypair, its own EGS serial), with the **same VAT number** in the SAN UID field and the branch name in OU.
- Each branch redeems its own OTP, gets its own Compliance CSID, runs its own six compliance-test documents, and gets its own Production CSID.
- Each branch maintains its **own independent ICV/PIH hash chain** — no cross-branch locking needed, no shared row, no serialization bottleneck.
- Matches ZATCA's actual intended "one CR/VAT, N EGS units" design, and scales cleanly: adding branch #6 never touches branches #1–5's certificates or chains.
- Slightly more onboarding work per branch (each needs its own Fatoora OTP round trip), but this is a one-time cost per branch, not a per-invoice cost.

Recommendation

Build the new project on

Model B

. Key the identity/credential table by BranchId from day one (not a singleton with a hard-coded fixed PK) — retrofitting per-branch identity onto a shared-singleton design later is a real migration (the reference implementation had to do exactly this split once, moving from "per-branch crypto columns" to "shared singleton" and would need to reverse that decision to reach Model B). Every place that reads "the ZATCA identity" (invoice submission, hash-chain increment, gating checks) must take a branchId parameter and never assume a single global identity.

What stays shared across branches vs. what's per-branch

FIELD	SHARED ACROSS BRANCHES?	NOTES
VAT registration number	Yes	One legal entity, one VAT number, embedded in every branch's CSR.
CR / Commercial Registration	Yes	Used as PartyIdentificationId (scheme CRN) on every invoice.
Fatoora portal account	Yes	One admin/owner login used to onboard every branch's CSR.
CSR / keypair / EGS serial	No — per branch	Each branch is its own EGS device.
Compliance & Production CSID	No — per branch	Each branch's certificate is independently issued.
ICV / PIH hash chain	No — per branch	Independent tamper-evident sequences.

Seller/branch display address	No — per branch	Street/building/postal fields feed the invoice UBL and the CSR's <code>registeredAddress</code> .
Phase2Enabled toggle / environment	Recommend per-branch, default inherited	Lets you roll production out branch-by-branch instead of flipping the whole company on at once.

5 Data model recommendation

Two entities, cleanly separated by concern — both keyed by branch under the recommended Model B:

```
ZatcaBranchIdentity (one row PER branch)
  BranchId          FK -> Branches
  Environment        sandbox | simulation | production
  Phase2Enabled      bool
  EgsSerial          string
  Csr                base64, nullable until step 1
  PrivateKey         encrypted at rest (data-protection API / KMS)
  CcsidRequestId / CcsidBinarySecurityToken / CcsidSecret (secret encrypted)
  PcsidRequestId / PcsidBinarySecurityToken / PcsidSecret (secret encrypted)
  LastIcv            int, this branch's own counter
  LastInvoiceHash    base64, this branch's own chain tip
  OnboardingStatus   not_started | csr_generated | compliance_csid_obtained | production_ready
  CreatedAt / UpdatedAt

ZatcaBranchSettings (display/UBL fields, one row per branch)
  BranchId          FK -> Branches
  VatRegistrationNumber
  SellerName
  StreetName / BuildingNumber / CitySubdivisionName / PostalZone
```

Why split identity from display settings

Display fields (seller name, address) get edited by store staff through a settings screen. Cryptographic identity must never be editable through a generic "edit settings" form and should be encrypted at rest with a dedicated key/protector, separate from ordinary application data. Keeping them as two tables (joined 1:1 by BranchId) makes it trivial to expose a secrets-scrubbed DTO to the frontend — return booleans (HasCsr, HasComplianceCsid, HasProductionCsid) instead of the raw key material.

6 Invoice generation & signing pipeline

6.1 Building the UBL 2.1 XML

Start from ZATCA's UBL 2.1 invoice template and fill it via XPath/DOM mutation (namespaces `cbc=CommonBasicComponents-2`, `cac=CommonAggregateComponents-2`):

- `cbc:InvoiceTypeCode` inner text = 388/381/383 (invoice/credit-note/debit-note); its name attribute = 0100000 (standard) or 0200000 (simplified) — **this attribute is the real B2B/B2C discriminator** used everywhere downstream, not a separate flag.
- `AdditionalDocumentReference[ID=ICV]` and `[ID=PIH]` carry the invoice counter and previous-invoice-hash.
- Credit/debit notes add an `InstructionNote` (KSA-10) and a `BillingReference` pointing at the original invoice (KSA-16); plain invoices must not carry a `BillingReference` node at all.
- **Customer/buyer party is only populated for standard (B2B) invoices.** A simplified/B2C invoice has no buyer VAT captured — that absence is exactly what should decide simplified vs. standard (see below), not a separate manually-set flag.
- Line items: quantity, line extension amount, per-line tax amount + rounding, item name, tax category percent, unit price. Apply any document-level discount once via `AllowanceCharge/LegalMonetaryTotal` — do not allocate it per line.

Decide simplified vs. standard from data, not from a flag

The cleanest signal: `isSimplified = string.IsNullOrEmpty(BuyerVatNumber)`. A POS/retail checkout that never captures a buyer VAT number should always produce simplified/B2C invoices; a manual invoice-creation path that captures a buyer VAT should produce standard/B2B ones.

6.2 Canonicalization & hashing

1. Apply ZATCA's documented XSLT transform to strip/normalize the invoice XML for hashing purposes.
2. Apply standard XML-DSig Canonicalization (C14N, `exclusive=false`, no comments) to the transformed XML.
3. `SHA-256(UTF8 bytes of the canonical XML) → base64` = the **invoice hash**. This value feeds both the QR code (tag 6) and becomes the PIH seed for the *next* invoice in that branch's chain.

6.3 Signing (simplified invoices)

Simplified invoices get a full XAdES signature block plus QR. The two most easily-missed details:

- **Certificate digest** for XAdES `CertDigest` is SHA-256 over the **UTF-8 bytes of the base64 text** of the certificate — not the decoded DER bytes. This is a genuine ZATCA-spec deviation from how X.509 cert digests normally work, and it is easy to get "structurally plausible but wrong" here.
- **The digital signature** is ECDSA-SHA256 computed **over the already-computed invoice-hash bytes** — effectively signing a hash-of-a-hash, matching how ZATCA's reference tooling calls `openssl_sign`, not a fresh signature over the raw canonical XML.
- Embed `SigningTime`, `CertDigest`, `IssuerSerial` (parsed straight out of the certificate's ASN.1 DER) into a hand-built `SignedProperties` block — exact whitespace matters because it gets hashed as literal text, not re-serialized.

Standard (B2B/clearance) invoices: don't skip signing

The reference implementation currently sends standard invoices through clearance *without* the XAdES signature/QR wrapping applied to simplified invoices — a known gap flagged in its own code comments, not a spec-compliant shortcut. ZATCA's clearance flow for standard invoices generally expects the same cryptographic signing as simplified ones.

Do not copy that particular shortcut into a new build

if real B2B clearance is in scope — sign standard invoices the same way as simplified ones before submitting to the clearance endpoint.

7 QR code generation

The QR payload is a Base64-encoded TLV (Tag-Length-Value) byte buffer, built only for simplified/B2C invoices (standard invoices don't get a QR in the flow described in §6, though per spec they generally should once fully compliant).

TAG	FIELD	SOURCE	ENCODING
1	Seller name	Supplier legal registration name	UTF-8 text
2	VAT registration number	Supplier tax scheme company ID	UTF-8 text
3	Timestamp	<code>{'{' }IssueDate{' }'}T{'{' }IssueTime{' }'}</code>	UTF-8 text
4	Invoice total (incl. VAT)	Legal monetary total — payable amount	UTF-8 text
5	VAT total	Tax total amount	UTF-8 text
6	Invoice hash	The base64 SHA-256 invoice hash from §6.2	UTF-8 text of the base64 <i>string</i> , not decoded bytes
7	Digital signature	The ECDSA signature from §6.3	UTF-8 text of the base64 <i>string</i> , not decoded bytes
8	Public key	<code>SubjectPublicKeyInfo</code> from the certificate DER	Raw DER bytes
9	Certificate signature	The issuing CA's signature over the certificate	Raw DER bytes

Rendering

The backend only needs to produce the base64 TLV string — it does **not**

need to rasterize a QR image. Render the QR client-side (e.g. a React `<QRCodeSVG value={'{' }zatcaQr{' }' } />` component, or the equivalent for the receiving platform) at the point of display: the staff POS screen, the self-checkout kiosk, and the printed receipt (thermal printers accept raw ESC/POS QR-code commands, so no image asset is needed there either).

Phase-1 fallback

If a branch isn't yet Phase-2 onboarded (or a ZATCA submission fails), fall back to a simpler 5-tag TLV (tags 1–5 only, no hash/signature/keys) built entirely client-side from data already on the receipt. This keeps receipts printable and QR-scannable even before/during an outage, while making clear (via invoice status) that the full Phase-2 signature hasn't been obtained yet.

8 Checkout integration & the reporting/clearance split

8.1 When to submit

Trigger ZATCA submission immediately after a sale is finalized/committed at checkout, gated on that **branch's** onboarding status:

```
if (branchIdentity.Phase2Enabled && branchIdentity.OnboardingStatus == "production_ready")
{
    // build + sign + submit
}
```

8.2 Synchronous vs. background submission

Submit **synchronously, inline in the checkout request** — not fire-and-forget via a background queue — so that the checkout response (and the printed receipt) can carry the real ZATCA-signed QR code rather than a client-side approximation. Wrap the DB write + external HTTP call in a transaction that row-locks the branch's identity row, so two concurrent checkouts on the same branch can't fork its ICV/PIH chain.

Never let a ZATCA failure block the sale

If the ZATCA API call fails (network error, gateway down, rejection), the invoice row should be left in a pending/error state and an admin notified — but

the checkout itself must still succeed

. A cashier ringing up a customer cannot be made to wait on, or fail because of, a government API's availability. Provide a manual "resubmit" action on an admin dashboard for anything left pending, since ZATCA still expects simplified invoices reported within 24 hours even if the original checkout-time submission failed.

8.3 Reporting vs. clearance

	SIMPLIFIED (B2C)	STANDARD (B2B)
Endpoint	POST /{{'{'}}env{'{'}}/invoices/reporting/single	POST /{{'{'}}env{'{'}}/invoices/clearance/single
Timing	Post-hoc; must occur within 24 hours of the sale (submit immediately at checkout regardless — don't rely on a queue that could exceed the SLA)	Real-time, blocking — must complete <i>before</i> the invoice is delivered to the buyer
Success signal	reportingStatus contains REPORTED	clearanceStatus contains CLEARED
Auth	Basic Auth using the Production CSID token/secret; header Clearance-Status: 1	

On acceptance, advance that branch's LastIcv/LastInvoiceHash to chain into the next invoice. On rejection, do **not** advance them — a rejected invoice must not break the hash chain for the next successful one.

9 API endpoint reference

ROUTE	PURPOSE
GET /zatca/settings/{''}branchId{''}}	Merged branch display settings + onboarding-status booleans, secrets scrubbed.
PUT /zatca/settings/{''}branchId{''}}	Update branch display fields + Phase2Enabled/environment.
POST /zatca/onboarding/{''}branchId{''}/csr	Step 1 — generate CSR/keypair for this branch.
POST /zatca/onboarding/{''}branchId{''}/compliance-csid	Step 3 — redeem OTP for this branch's Compliance CSID.
POST /zatca/onboarding/{''}branchId{''}/production-csid	Step 4–5 — run the six compliance tests, then request this branch's Production CSID.
GET /zatca/invoices?branchId=&status=	List invoices, branch-scoped, newest first.
POST /zatca/invoices	Create an invoice row (used by non-POS flows, e.g. manual B2B invoicing).
POST /zatca/invoices/{''}id{''}/submit	Manually (re)submit a pending/failed invoice — the retry path.
PATCH /zatca/invoices/{''}id{''}/status	Manual override of invoice status (admin correction tooling).

10 Gotchas & lessons learned (the expensive ones)

1. CSR "SN" field OID confusion — cost an entire debugging session

ZATCA's reference `openssl.cnf` uses the OpenSSL short name `SN` for the EGS-serial SAN attribute. In OpenSSL, `SN` is short for **surname**

(OID 2.5.4.4), not `serialNumber` (OID 2.5.4.5) — a deeply counter-intuitive abbreviation. A CSR built with a library's "SerialNumber" helper (OID 2.5.4.5) is byte-length-identical to a correct one and is silently rejected by the live gateway with a bare 400 "Invalid Request" and no field-level detail. The only way this surfaced was a byte-level diff against a known-good CSR from a reference implementation, isolating a single-digit OID difference inside the SAN's OCTET STRING payload.

If a CSR/onboarding rejection ever comes back as a generic "Invalid Request," suspect an OpenSSL short-name-vs-OID mismatch before anything else

— verify every custom OID against an actual accepted CSR, byte for byte, not against a library's named constant.

2. Certificate digest hashes the base64 TEXT, not the decoded DER

The XAdES `CertDigest` is SHA-256 of the UTF-8 bytes of the certificate's base64 *text* representation — not the decoded binary certificate. This is a real ZATCA spec deviation from how X.509 digests normally work elsewhere, and "looks correct" code review will not catch it.

3. EC domain parameters need named-curve encoding

When constructing the EC key material, use named domain parameters (e.g. BouncyCastle's `ECNamedDomainParameters`), not plain domain parameters — otherwise the public key serializes with an explicit-curve OID instead of the named `secp256k1` curve OID, which ZATCA's gateway also rejects.

4. QR tags 6 & 7 carry base64 text bytes, not decoded bytes

Unlike tags 8 and 9 (raw DER), the invoice-hash and signature TLV tags encode the *text* of their base64 representation as UTF-8 bytes — not the bytes you'd get from decoding that base64 string. Mixing these two conventions up produces a QR that scans fine structurally but fails ZATCA's own QR-verification tooling.

5. Only a live round trip counts as verification

None of the above bugs were visible from structural ASN.1 parsing, type-level checks, or "does this look like a valid X.509/XAdES document." Every one only surfaced by comparing byte-for-byte against a known-accepted reference, or by hitting ZATCA's actual sandbox gateway and reading the real response. Budget time for an actual sandbox round trip before considering onboarding "done" — visual code review of crypto code is not a reliable signal here.

6. Content-Type header must be bare

ZATCA's gateway wants `Content-Type: application/json` with no `charset` parameter appended — many HTTP client libraries add one by default and need to be told not to.

11 Frontend integration notes

- **Onboarding wizard** (admin/settings screen, per branch under Model B): Generate CSR → show the operator the base64 CSR to paste into the Fatoora portal → input for the OTP received back → "Get Compliance CSID" action → "Run Compliance Tests & Get Production CSID" action with a live pass/fail table for the six test documents.
- **Invoice dashboard**: list of submitted invoices with status (pending/reported or cleared/rejected/error), filterable by branch, with a manual retry action for anything not yet accepted.
- **Checkout / receipt QR**: render the server-provided TLV via a client-side QR component; fall back to the local Phase-1 5-tag TLV if the server didn't return one (branch not onboarded yet, or submission failed).
- **Compliance/VAT report**: a simple KPI view (taxable sales, VAT collected, count of reported/cleared vs. pending vs. errored invoices) is enough for day-to-day operational visibility into whether ZATCA submission is keeping up.

12 Build checklist for the new project

- ☐ Confirm VAT number, CR number, and Fatoora portal access are available before starting.
- ☐ Decide up front: Model B (per-branch CSID, recommended) vs. Model A (shared identity) — and key the identity table accordingly from the very first migration.
- ☐ Implement CSR generation with every SAN field from the §3 table, verified byte-for-byte against a known-accepted CSR before attempting a real OTP redemption.
- ☐ Encrypt private keys and CSID secrets at rest; never expose them through a settings API — return booleans only.
- ☐ Build the UBL invoice XML with the simplified/standard split driven by presence of a buyer VAT number.
- ☐ Implement canonicalization + hashing exactly per §6.2, and sign **both** simplified and standard invoices (don't replicate the reference implementation's standard-invoice signing gap).
- ☐ Build the 9-tag QR TLV per §7, with a Phase-1 5-tag fallback for un-onboarded branches.
- ☐ Wire checkout to submit synchronously, gated on that branch's `production_ready` status, with the sale never blocked by a ZATCA failure.
- ☐ Route simplified invoices to `reporting/single` and standard invoices to `clearance/single`; advance the branch's ICV/hash chain only on acceptance.
- ☐ Add a manual resubmit path and admin alerting for anything left pending/errored — there's no free automated retry queue in ZATCA's model, you must build one or handle it manually.
- ☐ Run a real sandbox round trip (CSR → OTP → compliance CSID → six compliance tests → production CSID → one real reported invoice) before calling onboarding done.

Prepared from a working reference implementation (Mimony Mart POS, .NET/C#) verified end-to-end against ZATCA's live sandbox gateway. Field names, OIDs, and endpoint paths reflect the ZATCA Developer Portal specification as implemented and tested as of July 2026 — confirm against ZATCA's current published documentation before production go-live, as government specifications are occasionally revised.